

2025-겨울학기 Dash Lab 학부연구인턴

Pruning 적용 MLP 가속기



중앙대학교 전자전기공학부 | 홍동의

01 Pruning&Quantization

- **Sensitivity Analysis**
 - Granularity: Fine-grained
 - Criterion: L1-norm
 - Ratio: 0%~95%까지 5%단위

```
def PSA(model, test_DL, DEVICE):
    weight_param_list = [p for p in model.named_parameters() if 'weight' in p[0]]
    layers=len(weight_param_list)
    org_acc=Test(model, test_DL, DEVICE)
    results = {}

    pruned_model = copy.deepcopy(model)
    params_dict = dict(pruned_model.named_parameters())

    for i in range(layers):
        target_name, original_param = weight_param_list[i]
        results[target_name]=[]
        weight_tensor=original_param.data.clone()

        # criterion
        l1_mag=torch.abs(weight_tensor)
        # granularity
        val,idx= torch.sort(l1_mag.flatten())

        for j in range(20):
            ratio=j/20.0
            n_prune = int(weight_tensor.numel() * ratio)

            # pruning 마스크 생성
            pruning_mask_flat=torch.ones_like(l1_mag.flatten(), dtype=torch.float32)
            pruning_mask_flat[idx[:n_prune]]=0
            pruning_mask=pruning_mask_flat.reshape(l1_mag.shape)

            with torch.no_grad():
                # pruning weight 생성
                pruned_weight=weight_tensor*pruning_mask
                # pruning 적용
                params_dict[target_name].copy_(pruned_weight)

            _, acc = Test(pruned_model, test_DL, DEVICE)
            results[target_name].append({'ratio': ratio, 'acc': acc})

        print(f'Progress: {100*(i*20+j)/(layers*20.0):.2f}%')

    with torch.no_grad():
        # pruning 복구
        params_dict[target_name].copy_(weight_tensor)

    return results,org_acc
```

01 Pruning&Quantization

- **Sensitivity Analysis**
 - Granularity: Fine-grained
 - Criterion: L1-norm
 - Ratio: 0%~95%까지 5%단위

```
def PSA(model, test_DL, DEVICE):
    weight_param_list = [p for p in model.named_parameters() if 'weight' in p[0]]
    layers=len(weight_param_list)
    org_acc=Test(model, test_DL, DEVICE)
    results = {}

    pruned_model = copy.deepcopy(model)
    params_dict = dict(pruned_model.named_parameters())

    for i in range(layers):
        target_name, original_param = weight_param_list[i]
        results[target_name]=[]
        weight_tensor=original_param.data.clone()

        # criterion
        l1_mag=torch.abs(weight_tensor)
        # granularity
        val,idx= torch.sort(l1_mag.flatten())

        for j in range(20):
            ratio=j/20.0
            n_prune = int(weight_tensor.numel() * ratio)

            # pruning 마스크 생성
            pruning_mask_flat=torch.ones_like(l1_mag.flatten(), dtype=torch.float32)
            pruning_mask_flat[idx[:n_prune]]=0
            pruning_mask=pruning_mask_flat.reshape(l1_mag.shape)

            with torch.no_grad():
                # pruning weight 생성
                pruned_weight=weight_tensor*pruning_mask
                # pruning 적용
                params_dict[target_name].copy_(pruned_weight)

            _, acc = Test(pruned_model, test_DL, DEVICE)
            results[target_name].append({'ratio': ratio, 'acc': acc})

        print(f'Progress: {100*(i*20+j)/(layers*20.0):.2f}%')

    with torch.no_grad():
        # pruning 복구
        params_dict[target_name].copy_(weight_tensor)

    return results,org_acc
```

```

ratios = {}
threshold_percent=10.0
min_acceptable_acc = org_acc - threshold_percent

for layer_name, data_list in results.items():
    if 'classifier' in layer_name:
        best_ratio = 0.0
        acc_at_best = org_acc
        for data in data_list:
            ratio = data['ratio']
            acc = data['acc']
            if acc >= min_acceptable_acc:
                best_ratio = ratio
                acc_at_best = acc
            else:
                break
        ratios[layer_name] = best_ratio
    print(f"{layer_name:<40} | {best_ratio:.2f} | {acc_at_best:.2f}%")

```

Python

classifier.0.weight	0.95	94.00%
classifier.3.weight	0.95	94.20%
classifier.6.weight	0.80	92.50%

```

pruned_rcorrect,pruned_acc=mm.Test(pruned_model, test_DL, DEVICE)
print(f"Test accuracy: {pruned_rcorrect}/{len(test_DL.dataset)} ({pruned_acc} %)")

```

Python

Test accuracy: 6829/10000 (68.3 %)

Final Test accuracy after fine-tuning: 9405/10000 (94.0 %)

```

-----Layer 1-----
S_in: 0.021054988726973534
Z_in: -128.0
S_out: 0.010501154698431492
Z_out: -128.0
S_w: 0.0010098598431795835
S_b: 2.1262587324599735e-05
q_w: torch.Size([512, 512])
q_b: torch.Size([512])
q_bias: torch.Size([512])
Scale_factor: 133.0
-----Layer 2-----
S_in: 0.010501154698431492
Z_in: -128.0
S_out: 0.0111627746373415
Z_out: -128.0
S_w: 0.000799719535280019
S_b: 8.397978490393143e-06
q_w: torch.Size([512, 512])
q_b: torch.Size([512])
q_bias: torch.Size([512])
Scale_factor: 49.0

```

Test accuracy 1: 9406/10000 (94.1 %)

■ CSR format?

1	7		
5		3	9
	2	8	
			6



Row pointer

Column
Index

Value

0	2	5	7	8			
0	1	0	2	3	1	2	3
1	7	5	3	9	2	8	6

02 Goals

```
input1=torch.round(input / layer_params[0]['s_in']) + layer_params[0]['z_in']
input1_0=input1[:,0:128]
print(input1_0)
```

✓ 0.0s Python

```
tensor([[ -127.,  -127.,  -116.,  -127.,  -122.,  -111.,  -128.,  -128.,  -119.,  -127.,
         -128.,  -122.,  -124.,  -128.,  -127.,  -118.,   -30.,  -128.,  -122.,  -127.,
        -128.,  -109.,  -128.,  -125.,   -86.,  -128.,  -128.,  -125.,  -119.,   -95.,
         -67.,  -126.,  -103.,  -128.,  -113.,  -117.,  -128.,  -128.,  -116.,  -118.,
        -110.,  -123.,  -127.,  -123.,  -128.,  -121.,  -128.,  -108.,  -124.,  -126.,
        -128.,   -95.,  -125.,  -128.,  -122.,  -123.,  -126.,  -102.,  -128.,  -128.,
        -128.,  -128.,  -115.,  -111.,  -120.,  -113.,   -98.,   -91.,  -100.,  -122.,
        -128.,  -128.,  -117.,  -128.,  -126.,  -127.,  -128.,  -111.,  -108.,  -128.,
         -94.,  -128.,  -127.,  -127.,  -127.,  -102.,   -94.,  -128.,  -127.,  -126.,
        -128.,  -107.,  -121.,  -128.,  -128.,  -128.,  -124.,  -128.,  -100.,   -48.,
        -123.,  -128.,  -119.,  -128.,  -127.,  -102.,  -128.,  -128.,   -93.,  -128.,
        -126.,  -128.,  -123.,  -128.,  -128.,   -98.,  -128.,  -126.,  -128.,  -128.,
         -82.,  -128.,  -128.,  -123.,  -128.,  -128.,   -91.,  -127.]],
        grad_fn=<SliceBackward0>)
```

```
q_w1=layer_params[0]['q_w']
q_bias1=layer_params[0]['q_bias']
scale_factor1=layer_params[0]['Scale_factor']
Z_out1=layer_params[0]['Z_out']

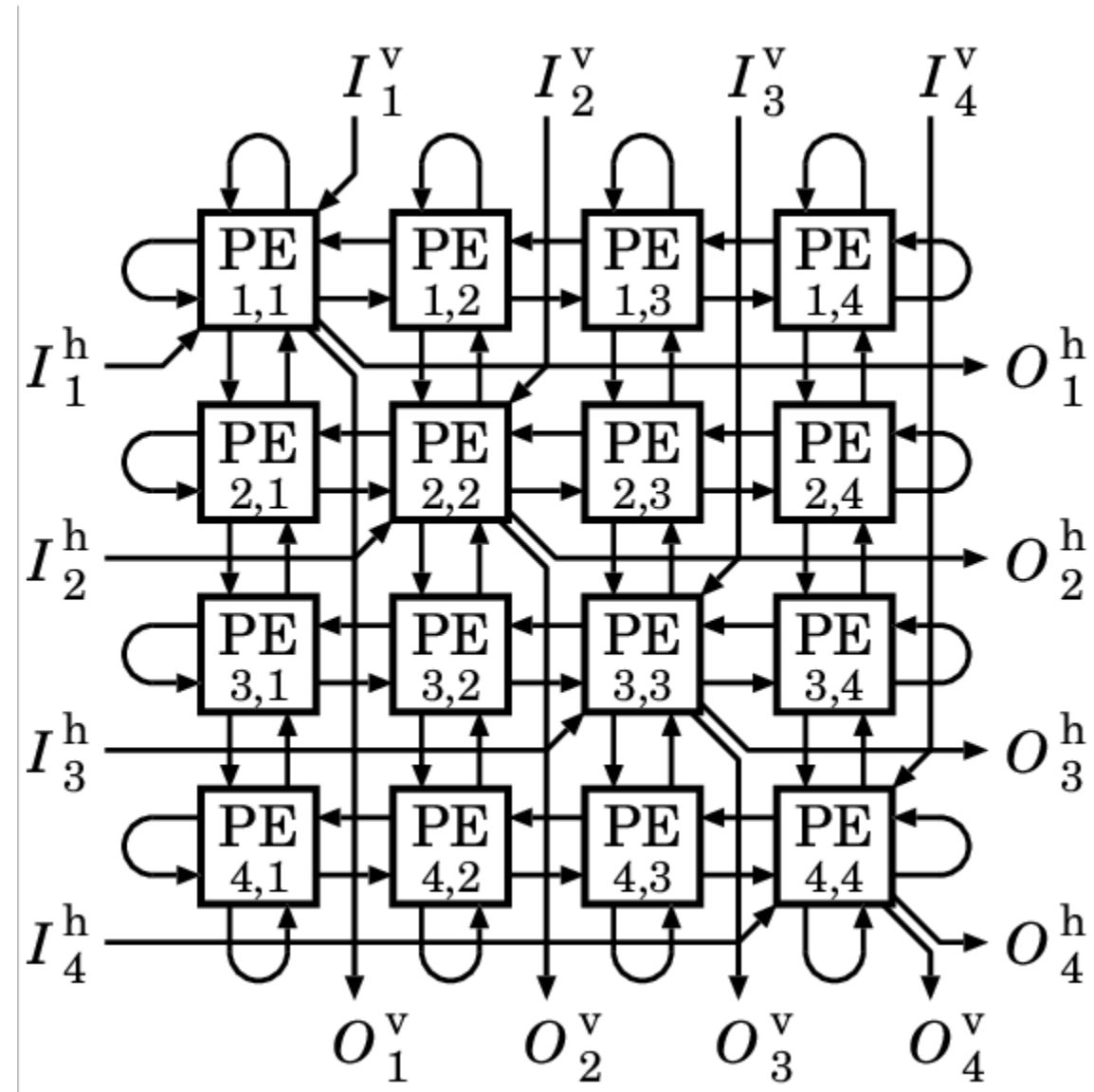
output_tmp1= quanted_sub_layers[0](input1)
output_tmp1=nn.ReLU()(output_tmp1)
output1= torch.floor(output_tmp1 * scale_factor1*(2**(-16))) + Z_out1

output1_0=output1[:,0:128]
print(output1_0)
```

✓ 0.0s Python

```
tensor([[ -65.,  -128.,   36.,  -128.,  -128.,  -29.,  -10.,  -128.,  -128.,  -128.,
        -127.,  -128.,  -128.,  -128.,  -25.,  -128.,  -128.,  -128.,  -128.,  -128.,
        -127.,  -125.,  -128.,  -128.,  -128.,  -128.,  -74.,  -106.,  -128.,  -13.,
        -115.,   -12.,  -126.,  -128.,  -128.,  -128.,   -3.,   25.,  -45., -104.,
        -128.,  -128.,  -62.,  -48.,  -128.,  -128.,  -82.,  -128.,  -53., -118.,
        -127.,  -128.,  -128.,  -13.,  -87., -116.,  -128.,  -128.,  -64.,  -128.,
        -48.,  -128., -118.,  -128.,   42., -102.,  -128.,  -128.,  -17.,  -54.,
         4.,  -127.,   -9.,  -128., -112.,  -27.,  -128.,  -128.,  -83.,  -128.,
        -128.,  -128.,  -128.,  -128.,  -128., -112.,  -128.,   -5.,  -128.,  -128.,
        -119.,  -55.,  -128.,  -50.,  -92.,   0.,  -128.,  -128.,  -128., -114.,
        -126., -110.,    3.,  -27.,  -128., -117.,  -128.,   -3.,  -128.,  -128.,
        -128.,  -128.,  -128.,  -81.,  -128.,  -128., -112., -121.,  -128.,  -26.,
        -128., -125.,  -128.,  -128.,   30.,  -128.,  -128.,  -56.]],
        grad_fn=<SliceBackward0>)
```

02 Goals



03 Previous work

...	...
VAL_5	IDX_5
VAL_4	IDX_4
VAL_3	IDX_3
VAL_2	IDX_2
VAL_1	IDX_1
VAL_0	IDX_0

32x32 matrix

PTR_32
...
PTR_4
PTR_3
PTR_2
PTR_1
PTR_0

Row pointer

COL_31	COL_30
...	...
COL_9	COL_8
COL_7	COL_6
COL_5	COL_4
COL_3	COL_2
COL_1	COL_0

Input vector

MAC 1개



OUT_32
...
OUT_4
OUT_3
OUT_2
OUT_1
OUT_0

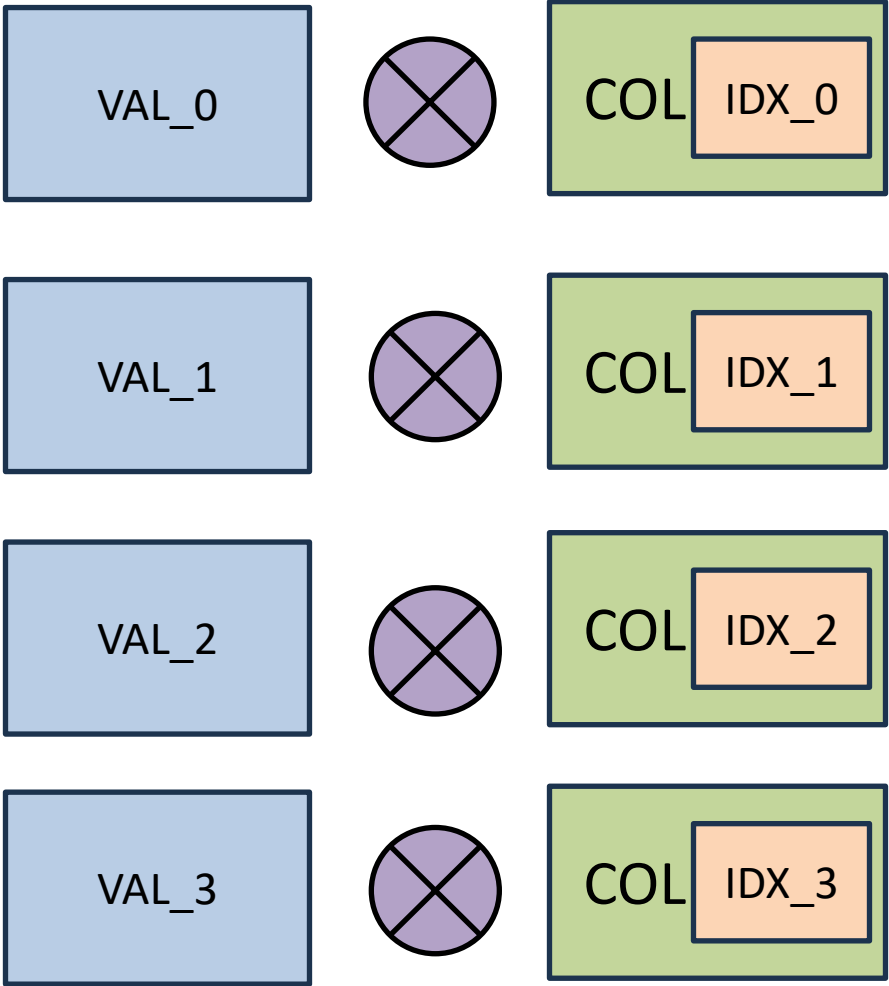
Output vector

04 First Plan

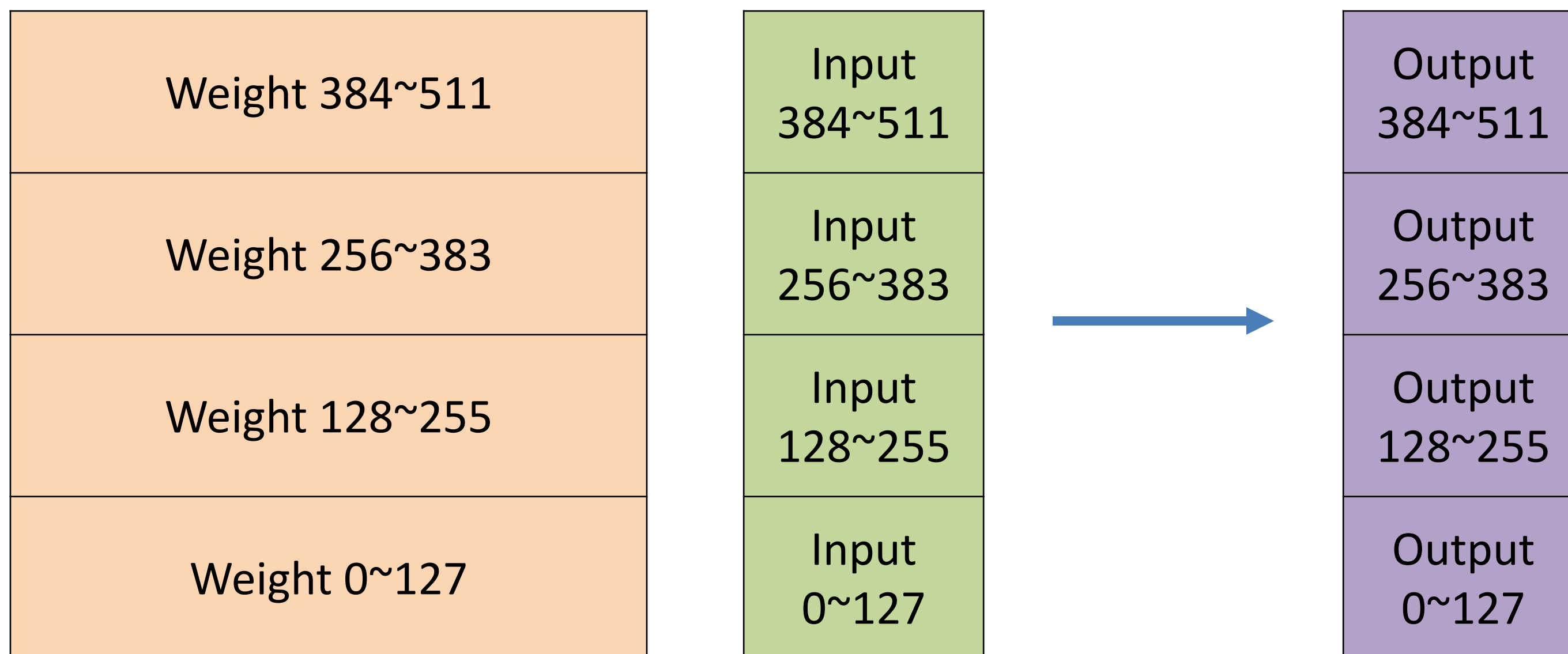
...	...	...	...
VAL_15	VAL_14	VAL_13	VAL_12
VAL_11	VAL_10	VAL_9	VAL_8
VAL_7	VAL_6	VAL_5	VAL_4
VAL_3	VAL_2	VAL_1	VAL_0

...	...	...	...
IDX_15	IDX_14	IDX_13	IDX_12
IDX_11	IDX_10	IDX_9	IDX_8
IDX_7	IDX_6	IDX_5	IDX_4
IDX_3	IDX_2	IDX_1	IDX_0

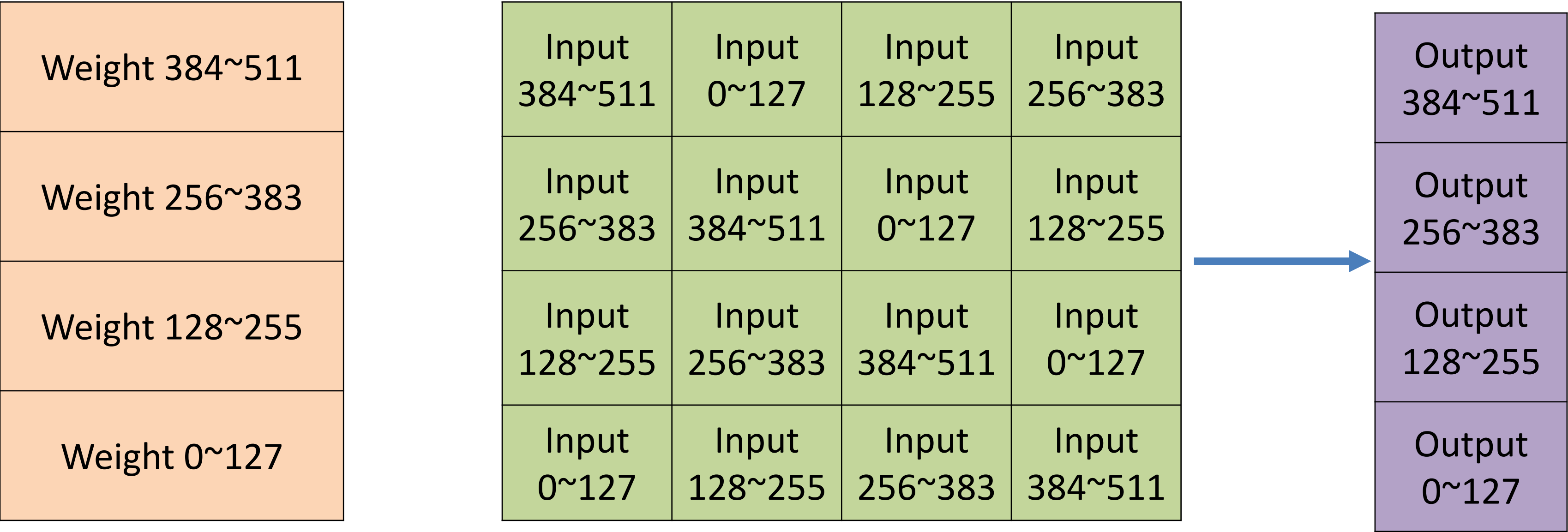
COL_512
...
COL_4
COL_3
COL_2
COL_1
COL_0



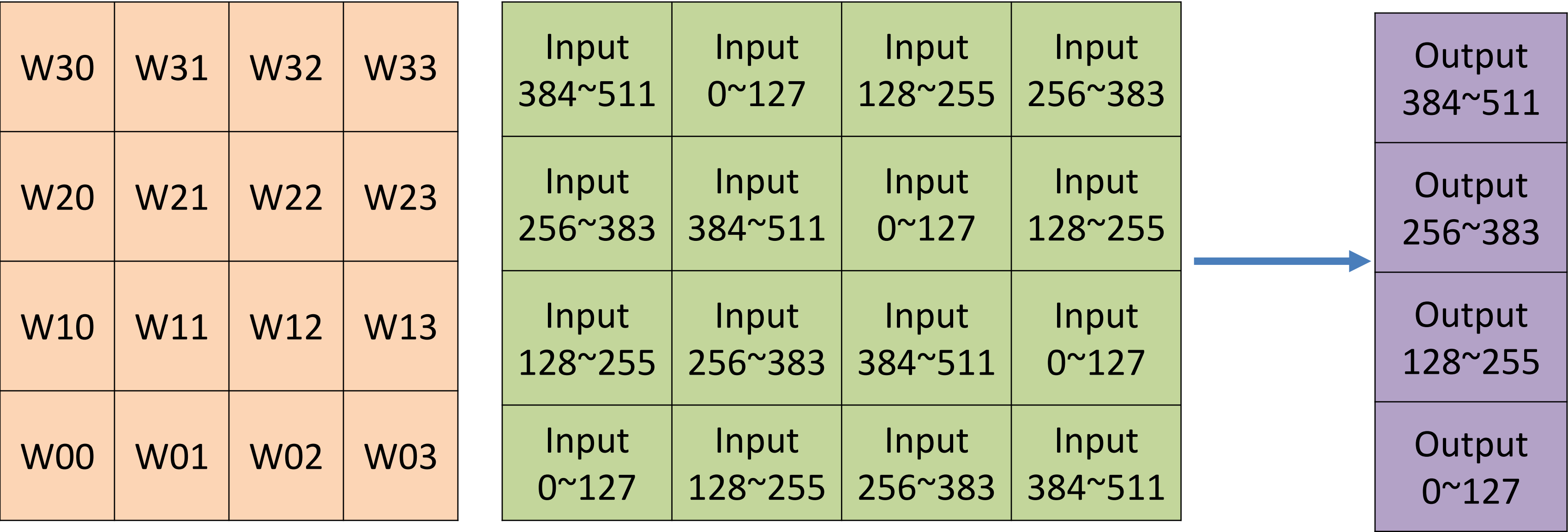
05 Submatrix



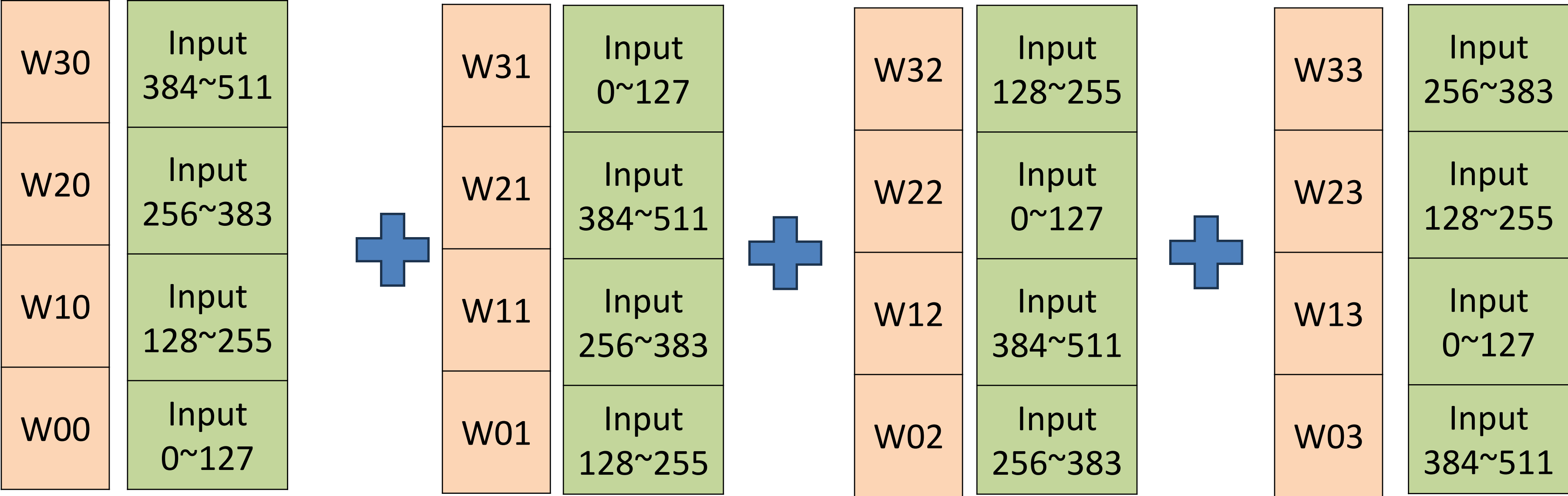
05 Submatrix



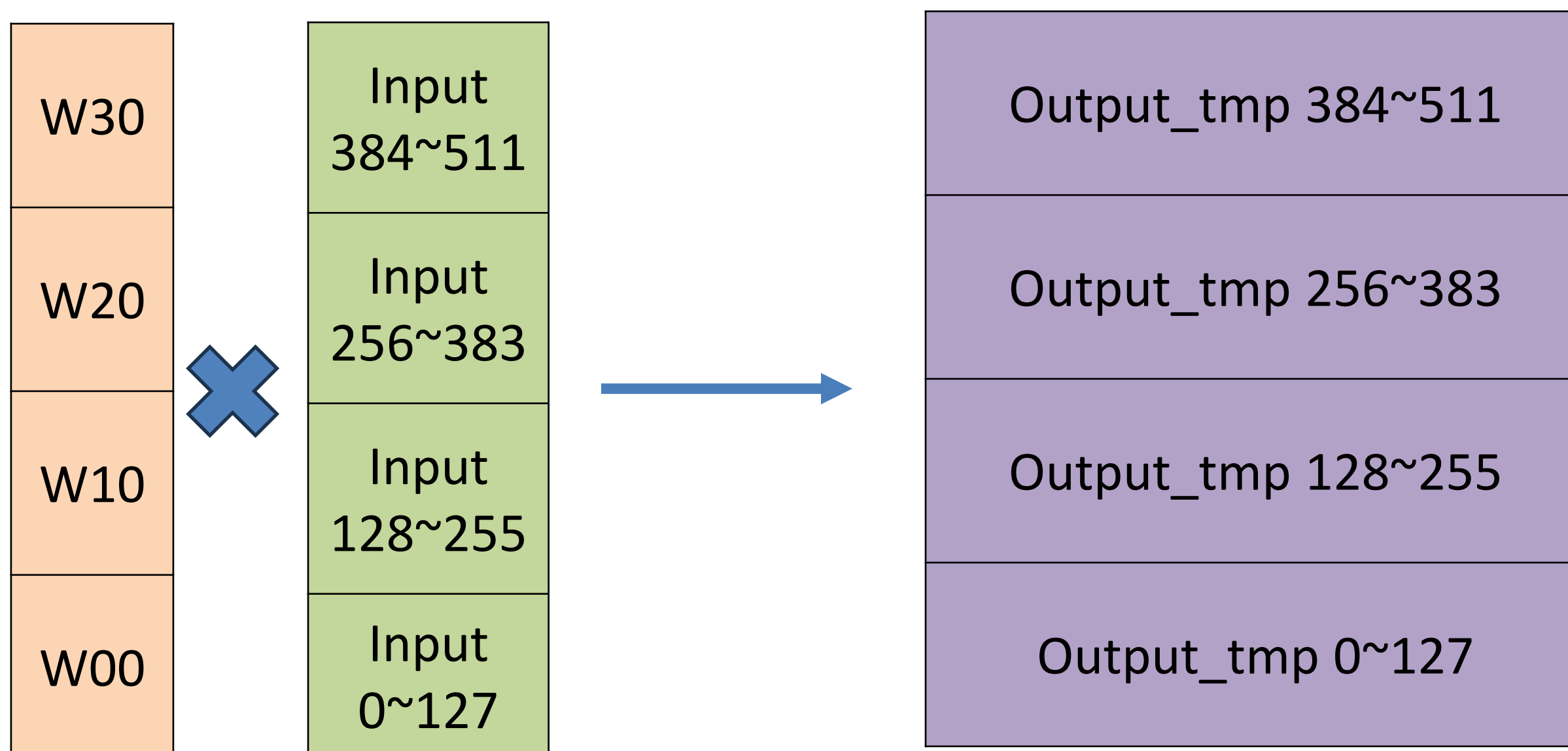
05 Submatrix



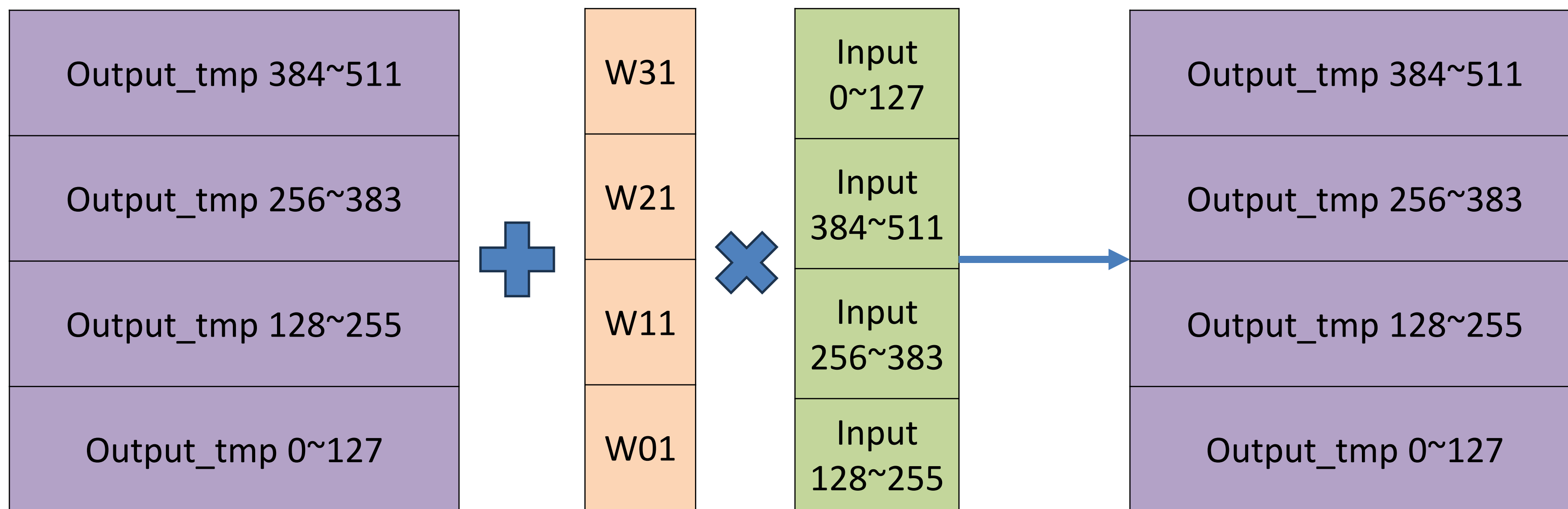
05 Submatrix



05 Submatrix



05 Submatrix



05 Submatrix

...	...
VAL_5	IDX_5
VAL_4	IDX_4
VAL_3	IDX_3
VAL_2	IDX_2
VAL_1	IDX_1
VAL_0	IDX_0

128x128 matrix

PTR_128
...
PTR_4
PTR_3
PTR_2
PTR_1
PTR_0

Row pointer

COL_127	COL_126
...	...
COL_9	COL_8
COL_7	COL_6
COL_5	COL_4
COL_3	COL_2
COL_1	COL_0

Input vector

MAC 1개
→

OUT_127
...
OUT_4
OUT_3
OUT_2
OUT_1
OUT_0

Output_tmp

06 128X128 SpMV

...	...
VAL_4	IDX_5
VAL_3	IDX_4
VAL_2	IDX_3
VAL_1	IDX_2
VAL_0	IDX_1
VAL_(-1)	IDX_0

128x128 matrix

ROW_num_127
...
ROW_num_4
ROW_num_3
ROW_num_2
ROW_num_1
ROW_num_0

Row num

COL_127
...
COL_4
COL_3
COL_2
COL_1
COL_0

Input vector

MAC 1개
→

OUT_127
...
OUT_4
OUT_3
OUT_2
OUT_1
OUT_0

Output_tmp

```

input1=torch.round(input / layer_params[0]['S_in']) + layer_params[0]['Z_in']
input1_0=input1[:,0:128]
print(input1_0)

```

✓ 0.0s

```

tensor([[ -127.,  -127.,  -116.,  -127.,  -122.,  -111.,  -128.,  -128.,  -119.,  -127.,
          -128.,  -122.,  -124.,  -128.,  -127.,  -118.,  -30.,  -128.,  -122.,  -127.,
          -128.,  -109.,  -128.,  -125.,  -86.,  -128.,  -128.,  -125.,  -119.,  -95.,
          -67.,  -126.,  -103.,  -128.,  -113.,  -117.,  -128.,  -128.,  -116.,  -118.,
          -110.,  -123.,  -127.,  -123.,  -128.,  -121.,  -128.,  -108.,  -124.,  -126.,
          -128.,  -95.,  -125.,  -128.,  -122.,  -123.,  -126.,  -102.,  -128.,  -128.,
          -128.,  -128.,  -115.,  -111.,  -120.,  -113.,  -98.,  -91.,  -100.,  -122.,
          -128.,  -128.,  -117.,  -128.,  -126.,  -127.,  -128.,  -111.,  -108.,  -128.,
          -94.,  -128.,  -127.,  -127.,  -127.,  -102.,  -94.,  -128.,  -127.,  -126.,
          -128.,  -107.,  -121.,  -128.,  -128.,  -128.,  -124.,  -128.,  -100.,  -48.,
          -123.,  -128.,  -119.,  -128.,  -127.,  -102.,  -128.,  -128.,  -93.,  -128.,
          -126.,  -128.,  -123.,  -128.,  -128.,  -98.,  -128.,  -126.,  -128.,  -128.,
          -82.,  -128.,  -128.,  -123.,  -128.,  -128.,  -91.,  -127.]],
      grad_fn=<SliceBackward0>)

```

memory[0:127][7:0]	-127,-127,-116,-127,-122,-111,-128,-128,-118,-127,-128,-122,-
> [1][7:0]	-127
> [2][7:0]	-116
> [3][7:0]	-127
> [4][7:0]	-122
> [5][7:0]	-111
> [6][7:0]	-128
> [7][7:0]	-128
> [8][7:0]	-118
> [9][7:0]	-127
> [10][7:0]	-128
> [11][7:0]	-122
> [12][7:0]	-124
> [13][7:0]	-128
> [14][7:0]	-127
> [15][7:0]	-118
> [16][7:0]	-30
> [17][7:0]	-128
> [18][7:0]	-122
> [19][7:0]	-127
> [20][7:0]	-128
> [21][7:0]	-109
> [22][7:0]	-128
> [23][7:0]	-125
> [24][7:0]	-86
> [25][7:0]	-128
> [26][7:0]	-128
> [27][7:0]	-125
> [28][7:0]	-119
> [29][7:0]	-95

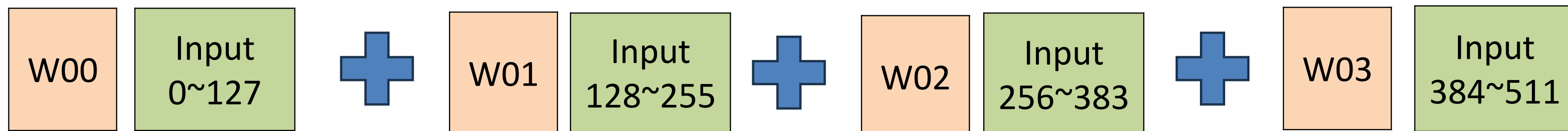
```
output_tmp1_00 = input1_splits[0].to(DEVICE) @ q_w1_splits[0][0].t().to(DEVICE)
print(output_tmp1_00[0][:])
```

✓ 0.0s

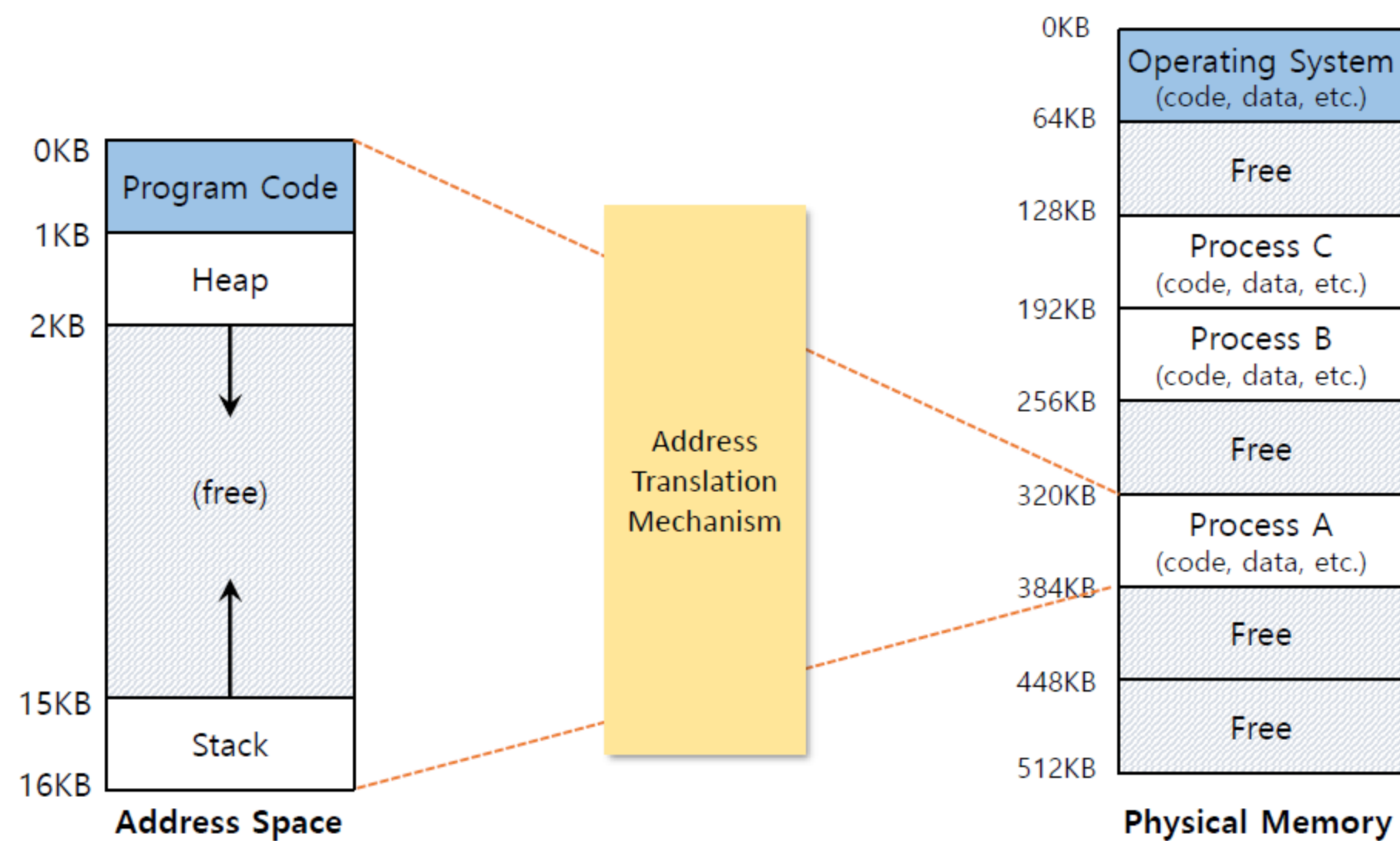
```
tensor([ -11698.,      0., -52212.,      0., -43285., -57384., -59922.,
          0.,      0.,      0., -82409., -93144., -75860.,      0.,
        -85001.,      0.,      0.,      0., -45837.,      0., -37148.,
       -102886.,      0.,      0., -52245., -71496., -35023., -56569.,
        -2944., -37373., -33384., -29542., -50287.,      0.,      0.,
          0., -27698., -49789., -18082., -80200., -53555.,      0.,
       -21870., -39985.,      0., -65634.,      0.,      0., -10094.,
       -50196., -46491.,      0.,      0., -59254., -25609., -57848.,
          0.,      0., -92272.,      0., -61823.,      0., -14359.,
       -43237., -30334., -13770., -39427.,      0., -61160., -21876.,
       -66502., -69697., -18734.,      0., -46854., -47048.,      0.,
          0., -11114.,      0.,      0.,      0.,      0.,      0.,
          0., -45858.,      0., -27891.,      0.,      0., -22798.,
       -28445.,      0., -30509., -38906., -60871.,      0., -45008.,
          0., -71206., -58333., -58722., -51618., -74080.,      0.,
       -34105.,      0., -56204.,      0.,      0., -38713.,      0.,
       -35907., -37171.,      0.,      0., -24835., -54496.,      0.,
       -21810., -68809., -33716., -38927.,      0., -70938.,      0.,
          0., -61342.], grad_fn=<SliceBackward0>)
```

memory[0:127][31:0]	-11698,0,-52212,0,-43285,-57384,-59922,0,0,0,-82409,-93144,-
> [0][31:0]	-11698
> [1][31:0]	0
> [2][31:0]	-52212
> [3][31:0]	0
> [4][31:0]	-43285
> [5][31:0]	-57384
> [6][31:0]	-59922
> [7][31:0]	0
> [8][31:0]	0
> [9][31:0]	0
> [10][31:0]	-82409
> [11][31:0]	-93144
> [12][31:0]	-75860
> [13][31:0]	0
> [14][31:0]	-84944
> [15][31:0]	0
> [16][31:0]	0
> [17][31:0]	0
> [18][31:0]	-45837
> [19][31:0]	0
> [20][31:0]	-37128
> [21][31:0]	-102886
> [22][31:0]	0
> [23][31:0]	0
> [24][31:0]	-52245
> [25][31:0]	-71496
> [26][31:0]	-35023
> [27][31:0]	-56569
> [28][31:0]	-2944

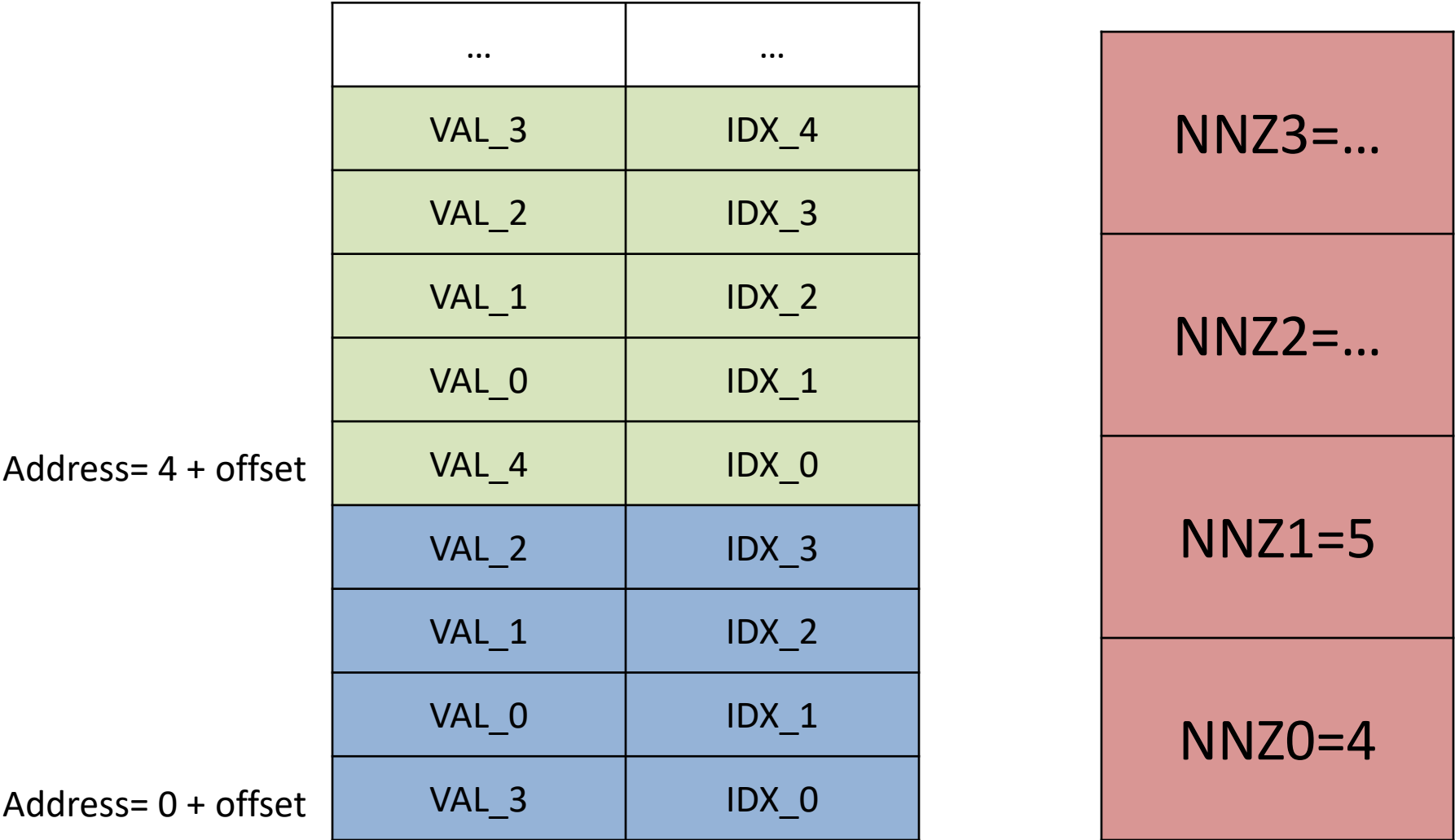
07 128X512 SpMV



07 Memory Virtualization



07 128X512 SpMV



```

mac_128x128 u_mac_128x128 (
    .i_clk(i_clk),
    .i_rst_n(i_rst_n),
    .i_start(i_start),
    .o_done(o_done),
    .i_quarter_cnt(i_quarter_cnt),
    .i_nnz(i_nnz),
    //weight memory interface(data)-read_only
    .o_addr_wd(o_addr_wd_virt),
    .o_en_wd(),
    .i_data_wd(i_data_wd),
    //weight memory interface(row num)-read_only
    .o_addr_row(o_addr_row_virt),
    .o_en_row(),
    .i_data_row(i_data_row),
    //q_bias memory interface-read_only
    .o_addr_bias(o_addr_bias_r),
    .o_en_bias(),
    .i_data_bias(i_data_bias),
    //input feature memory interface(col)-read_only
    .o_addr_in(o_addr_in_virt),
    .o_en_in(),
    .i_data_in(i_data_in),
    //output_tmp feature memory interface-read_and_write
    .o_addr_tmp(o_addr_tmp_1),
    .o_en_tmp(),
    .o_we_tmp(o_we_tmp_1),
    .o_data_tmp(o_data_tmp_1),
    .i_data_tmp(i_data_tmp)
);

```

```

wire [13:0] o_addr_wd_virt;
assign o_addr_wd = {1'b0, o_addr_wd_virt} + {i_addr_base};

```

```
output_tmp1_00 = input1_splits[0].to(DEVICE) @ q_w1_splits[0][0].t().to(DEVICE)
output_tmp1_01 = input1_splits[1].to(DEVICE) @ q_w1_splits[0][1].t().to(DEVICE) + output_tmp1_00[0].to(DEVICE)
output_tmp1_02 = input1_splits[2].to(DEVICE) @ q_w1_splits[0][2].t().to(DEVICE) + output_tmp1_01[0].to(DEVICE)
output_tmp1_03 = input1_splits[3].to(DEVICE) @ q_w1_splits[0][3].t().to(DEVICE) + output_tmp1_02[0].to(DEVICE)
print(output_tmp1_03[0][:])
```

✓ 0.0s Python

```
tensor([-143843.,      0., -144160.,      0., -149844., -205033., -243321.,
         0.,      0.,      0., -224004., -302538., -275966.,      0.,
        -198881.,      0.,      0.,      0., -140685.,      0., -120633.,
        -226409.,      0.,      0., -253722., -229775., -195181., -218287.,
        -11090., -153097., -140020., -174507., -180750.,      0.,      0.,
         0., -201591., -190112., -109285., -214621., -175633.,      0.,
        -192736., -175216.,      0., -245009.,   -7245.,      0., -172971.,
        -172889., -201557.,      0.,      0., -216946., -159301., -113821.,
         0.,      0., -217970.,      0., -171104.,      0.,  -71350.,
        -176339., -166632., -130387., -156552.,      0., -142003., -135999.,
        -193072., -225968., -127828.,      0.,  -90777., -199816.,      0.,
         0., -105192.,      0.,      0.,      0.,      0.,      0.,
         0., -131151.,      0., -107372.,      0.,      0., -182616.,
        -121443.,      0., -142333., -182902., -199949.,      0., -165258.,
         0., -192121., -179166., -118221., -171309., -204940.,      0.,
        -53097.,      0., -207207.,      0.,      0., -139962.,      0.,
        -177993., -185097.,      0.,      0., -131697., -147815.,      0.,
        -157856., -250728., -156631., -137758.,      0., -273635.,      0.,
         0., -217468.], grad_fn=<SliceBackward0>)
```

memory[0:127][31:0]	-143699,0,-143824,0,-149886,-204752,-243051,0,0,0,-224004,-
> [0][31:0]	-143699
> [1][31:0]	0
> [2][31:0]	-143824
> [3][31:0]	0
> [4][31:0]	-149886
> [5][31:0]	-204752
> [6][31:0]	-243051
> [7][31:0]	0
> [8][31:0]	0
> [9][31:0]	0
> [10][31:0]	-224004
> [11][31:0]	-302572
> [12][31:0]	-275966
> [13][31:0]	0
> [14][31:0]	-198686
> [15][31:0]	0
> [16][31:0]	0
> [17][31:0]	0
> [18][31:0]	-140720
> [19][31:0]	0
> [20][31:0]	-120645
> [21][31:0]	-226409
> [22][31:0]	0
> [23][31:0]	0
> [24][31:0]	-253834
> [25][31:0]	-229870
> [26][31:0]	-195029
> [27][31:0]	-218251
> [28][31:0]	-11090

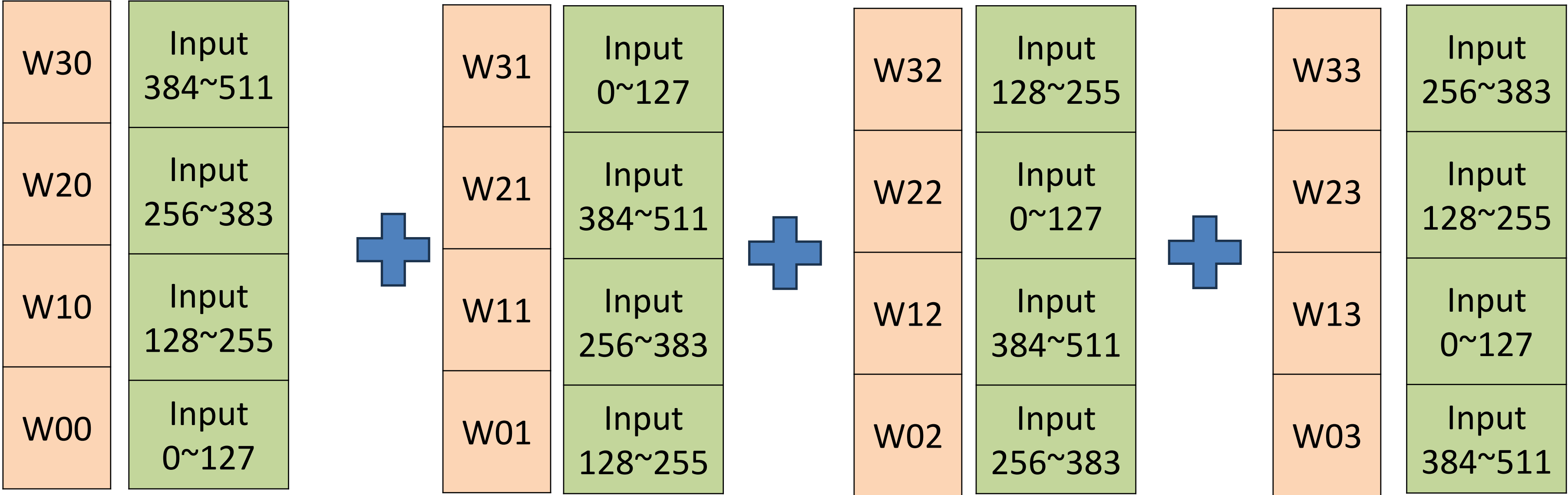

```
output_tmp1_00 = input1_splits[0].to(DEVICE) @ q_w1_splits[0][0].t().to(DEVICE)
output_tmp1_01 = input1_splits[1].to(DEVICE) @ q_w1_splits[0][1].t().to(DEVICE) + output_tmp1_00[0].to(DEVICE)
output_tmp1_02 = input1_splits[2].to(DEVICE) @ q_w1_splits[0][2].t().to(DEVICE) + output_tmp1_01[0].to(DEVICE)
output_tmp1_03 = input1_splits[3].to(DEVICE) @ q_w1_splits[0][3].t().to(DEVICE) + output_tmp1_02[0].to(DEVICE)
output_tmp1_fin1= output_tmp1_03 + q_bias1_splits[0].to(DEVICE)
output_tmp1_fin2=nn.ReLU()(output_tmp1_fin1)
output_tmp1_fin3= torch.floor(output_tmp1_fin2 * scale_factor1*(2**(-16))) + Z_out1
print(output_tmp1_fin3[0][:])
```

✓ 0.0s Python

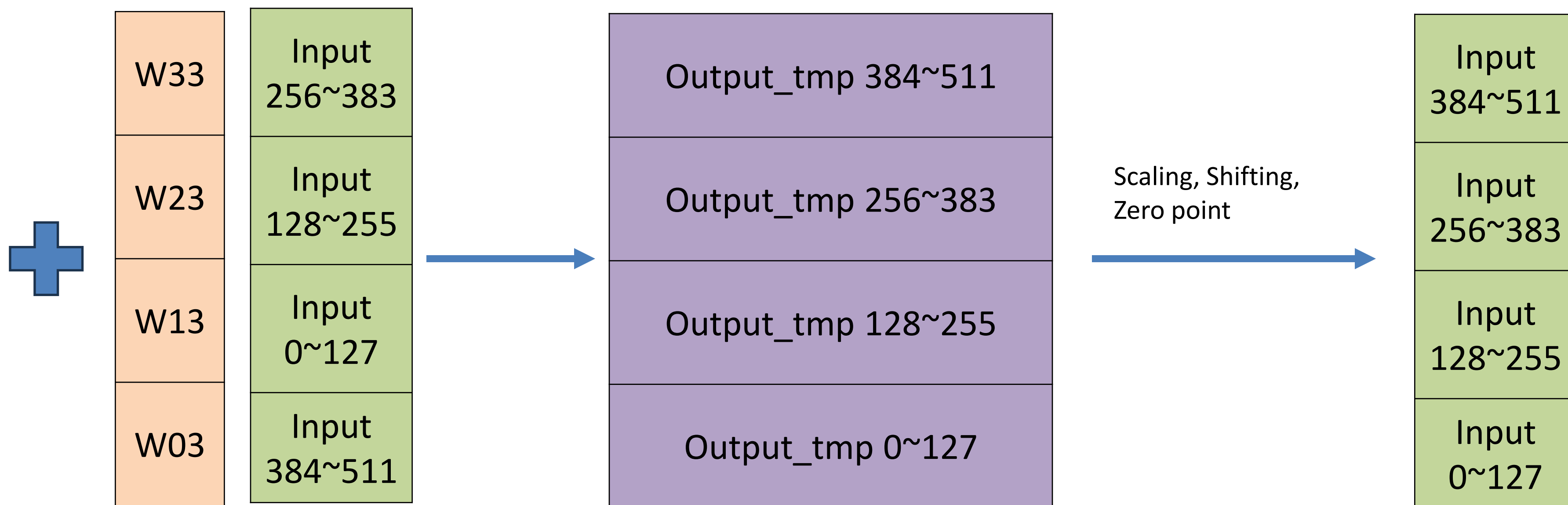
```
tensor([ -65., -128.,  36., -128., -128., -29., -10., -128., -128., -128.,
        -127., -128., -128., -128., -25., -128., -128., -128., -128., -128.,
        -127., -125., -128., -128., -128., -128., -74., -106., -128., -13.,
        -115., -12., -126., -128., -128., -128., -3.,  25., -45., -104.,
        -128., -128., -62., -48., -128., -128., -82., -128., -53., -118.,
        -127., -128., -128., -13., -87., -116., -128., -128., -64., -128.,
        -48., -128., -118., -128.,  42., -102., -128., -128., -17., -54.,
         4., -127., -9., -128., -112., -27., -128., -128., -83., -128.,
        -128., -128., -128., -128., -128., -112., -128., -5., -128., -128.,
        -119., -55., -128., -50., -92.,  0., -128., -128., -128., -114.,
        -126., -110.,  3., -27., -128., -117., -128., -3., -128., -128.,
        -128., -128., -128., -81., -128., -128., -112., -121., -128., -26.,
        -128., -125., -128., -128.,  30., -128., -128., -56.],
      grad_fn=<SliceBackward0>)
```

memory[0:127][7:0]	-66,-128,33,-128,-128,-30,-12,-128,-128,-128,-127,-128,-128,-128
> [0][7:0]	-66
> [1][7:0]	-128
> [2][7:0]	33
> [3][7:0]	-128
> [4][7:0]	-128
> [5][7:0]	-30
> [6][7:0]	-12
> [7][7:0]	-128
> [8][7:0]	-128
> [9][7:0]	-128
> [10][7:0]	-127
> [11][7:0]	-128
> [12][7:0]	-128
> [13][7:0]	-128
> [14][7:0]	-27
> [15][7:0]	-128
> [16][7:0]	-128
> [17][7:0]	-128
> [18][7:0]	-128
> [19][7:0]	-128
> [20][7:0]	-127
> [21][7:0]	-125
> [22][7:0]	-128
> [23][7:0]	-128
> [24][7:0]	-128
> [25][7:0]	-128
> [26][7:0]	-75
> [27][7:0]	-107
> [28][7:0]	-128

07 512X512 SpMV



07 512X512 SpMV



```
output_tmp1= input1@layer_params[0]['q_w'].T
print(output_tmp1[0][:128])
```

✓ 0.0s

```
tensor([-143843.,      0., -144160.,      0., -149844., -205033., -243321.,
         0.,      0.,      0., -224004., -302538., -275966.,      0.,
        -198881.,      0.,      0.,      0., -140685.,      0., -120633.,
        -226409.,      0.,      0., -253722., -229775., -195181., -218287.,
        -11090., -153097., -140020., -174507., -180750.,      0.,      0.,
         0., -201591., -190112., -109285., -214621., -175633.,      0.,
        -192736., -175216.,      0., -245009.,  -7245.,      0., -172971.,
        -172889., -201557.,      0.,      0., -216946., -159301., -113821.,
         0.,      0., -217970.,      0., -171104.,      0.,  -71350.,
        -176339., -166632., -130387., -156552.,      0., -142003., -135999.,
        -193072., -225968., -127828.,      0.,  -90777., -199816.,      0.,
         0., -105192.,      0.,      0.,      0.,      0.,      0.,
         0., -131151.,      0., -107372.,      0.,      0., -182616.,
        -121443.,      0., -142333., -182902., -199949.,      0., -165258.,
         0., -192121., -179166., -118221., -171309., -204940.,      0.,
        -53097.,      0., -207207.,      0.,      0., -139962.,      0.,
        -177993., -185097.,      0.,      0., -131697., -147815.,      0.,
        -157856., -250728., -156631., -137758.,      0., -273635.,      0.,
         0., -217468.], grad_fn=<SliceBackward0>)
```

memory[0:127][31:0]	-159444,0,-148577,0,-114859,-204299,-246311,0,0,0,-236778,-318
> [0][31:0]	-159444
> [1][31:0]	0
> [2][31:0]	-148577
> [3][31:0]	0
> [4][31:0]	-114859
> [5][31:0]	-204299
> [6][31:0]	-246311
> [7][31:0]	0
> [8][31:0]	0
> [9][31:0]	0
> [10][31:0]	-236778
> [11][31:0]	-318953
> [12][31:0]	-243392
> [13][31:0]	0
> [14][31:0]	-220925
> [15][31:0]	0
> [16][31:0]	0
> [17][31:0]	0
> [18][31:0]	-125912
> [19][31:0]	0
> [20][31:0]	-126662
> [21][31:0]	-220303

```
output_tmp1= input1@layer_params[0]['q_w'].T
output_tmp1=output_tmp1 + layer_params[0]['q_bias']
output_fin1=nn.ReLU()(output_tmp1)
output_fin2= torch.floor(output_fin1 * layer_params[0]['Scale_factor']*(2**(-16))) + layer_params[0]['Z_out']
print(output_fin2[0][:128])
```

✓ 0.0s Python

```
tensor([ -66., -128.,  34., -128., -128., -30., -11., -128., -128., -128.,
        -127., -128., -128., -128., -27., -128., -128., -128., -128., -128.,
        -128., -126., -128., -128., -128., -128., -76., -108., -128., -15.,
        -116., -14., -126., -128., -128., -128., -6., 23., -46., -106.,
        -128., -128., -64., -50., -128., -128., -82., -128., -54., -119.,
        -128., -128., -128., -15., -87., -117., -128., -128., -66., -128.,
        -49., -128., -119., -128., 40., -103., -128., -128., -18., -55.,
         2., -128., -11., -128., -112., -28., -128., -128., -84., -128.,
        -128., -128., -128., -128., -128., -113., -128., -6., -128., -128.,
        -119., -55., -128., -51., -94., -2., -128., -128., -128., -115.,
        -127., -111.,  1., -28., -128., -117., -128., -5., -128., -128.,
        -128., -128., -128., -83., -128., -128., -113., -121., -128., -28.,
        -128., -125., -128., -128., 27., -128., -128., -59.],
       grad_fn=<SliceBackward0>)
```

memory[0:127][7:0]	-97,-128,24,-128,-59,-30,-18,-128,-128,-128,-128,-128,-65,-128,-71,-1
> [0][7:0]	-97
> [1][7:0]	-128
> [2][7:0]	24
> [3][7:0]	-128
> [4][7:0]	-59
> [5][7:0]	-30
> [6][7:0]	-18
> [7][7:0]	-128
> [8][7:0]	-128
> [9][7:0]	-128
> [10][7:0]	-128
> [11][7:0]	-128
> [12][7:0]	-65
> [13][7:0]	-128
> [14][7:0]	-71
> [15][7:0]	-128
> [16][7:0]	-128
> [17][7:0]	-128
> [18][7:0]	-106
> [19][7:0]	-128
> [20][7:0]	-128
> [21][7:0]	-113

발표를 들어주셔서

감사합니다

